

Security & Access Document

Implementation-ready specification for a Pomodoro productivity application inspired by the functional model of pomodorotimer.online. Revised scope: anonymous, local-first, no login/signup, no user authentication.

1. Security Model

The product has no login, signup, account, user identity, access token, session cookie, or private cloud database in the initial release. The main security boundary is the browser origin and the user's local device.

2. Authentication

Authentication requirement: NONE. Do not add Supabase Auth, OAuth, magic links, passwords, or account UI to the initial scope.

3. Authorization

There are no multi-user server resources requiring user-level authorization or RLS. Browser storage is origin-scoped. Application code must nevertheless validate all local records before using them.

4. Data Classification

Data	Location	Server transmission
Timer state	localStorage	No
Settings	localStorage	No
Tasks	IndexedDB	No
Focus sessions/statistics	IndexedDB	No
Custom backgrounds	IndexedDB	No
Public bundled assets	Static/CDN	Downloaded to client
Optional privacy-safe analytics	Analytics provider	Only if deliberately enabled

5. Content Security

- Set a restrictive Content-Security-Policy.
- Use HTTPS and HSTS in production.
- Set X-Content-Type-Options: nosniff.
- Use a restrictive Referrer-Policy and Permissions-Policy.
- Avoid dangerouslySetInnerHTML for task/user-entered text.
- Pin and review dependencies; run vulnerability checks in CI.
- Restrict external media origins to explicitly approved domains.

6. Custom Image Security

- Accept only supported image MIME types and verify decoded image content.

- Set a maximum input size and maximum number of saved custom backgrounds.
- Re-encode/compress images client-side where practical.
- Do not execute SVG/HTML as user backgrounds in the initial release.
- Do not upload anonymous custom images to a server.

7. Notifications and Permissions

Request notification or Picture-in-Picture capabilities only in response to clear user actions and explain why permission is needed. The timer must remain usable when permissions are denied.

8. Storage Failure Handling

- Handle IndexedDB unavailable/quota errors without crashing the timer.
- Warn when custom images cannot be stored because quota is exceeded.
- Use atomic transactions for related writes.
- Never assume persisted JSON is trustworthy; parse and validate it.

9. Privacy

No personal productivity history should be transmitted by default. If analytics are introduced, minimize collection, avoid task titles/content, document the provider and retention behavior, and provide any consent/control required by applicable law.

10. Threat Validation

- Test XSS payloads in task title/description fields.
- Test malformed localStorage and IndexedDB records.
- Test oversized/invalid background uploads.
- Test third-party resource failure.
- Verify production headers.
- Verify no authentication endpoints or credentials are unnecessarily shipped.